Sentinel-ECU: A Smart Security Gateway for Protecting In-Vehicle CAN Networks

Malluri Sai Raghu (2300069038) · Devavarapu Ajaykumar (2300069039) · Jirayithi Mohammad Gouse (2300069041)
Name of the Supervisor: Dr. T. Vijay Muni
Department of Electrical and Electronics Engineering

Introduction

Objective of our work

Build a gateway ECU that sits between two CAN segments, checks every frame crossing from the exposed side to the protected side, and responds in proportion to the risk: log it, warn, drop it, or ask the vehicle's safety task for a degraded mode. The gateway never commands the brakes or steering itself.

Origin of our proposal

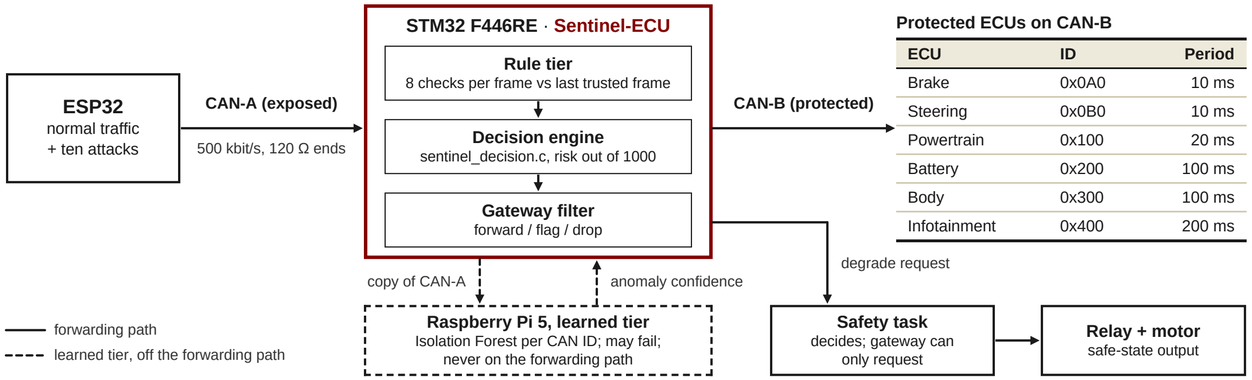
CAN has no sender authentication. Any node can transmit any ID, and every ECU trusts what it receives. UN R155 and ISO/SAE 21434 now require vehicles to detect intrusions. Yet most published CAN detectors stop at a label: they are rarely run on embedded hardware, and they seldom decide what the vehicle should do next. A false alarm that triggers an emergency stop is a safety failure in itself, so the response has to depend on how sure the gateway is and how critical the target function is.

Methods

Methods and Materials

- **Simulated bus.** Six ECUs with real periods (10 to 200 ms) and 3 % timing jitter. Every frame carries a rolling counter (byte 6) and a checksum (byte 7).
- **Attacks.** Ten attacks, two per class (spoofing, replay, flooding, fuzzing, value manipulation). Each ran 5 times for 12 s with different seeds.
- **Rule tier (planned for the STM32).**
  - Eight checks per frame against the last trusted frame on that ID: known ID, length, checksum, timing, flood rate, counter, exact copy, plausibility.
  - Only frames that pass become the new reference.
- **Learned tier (Raspberry Pi 5).** One Isolation Forest per CAN ID over six window features, trained on 8 × 30 s of clean traffic only, plus a learned feature envelope.
- **Decision engine.** sentinel_decision.c, integer-only C, called directly by the simulator.
  - Risk = attack weight × function weight × confidence, out of 1000.
  - Thresholds: WARN 150, FILTER 400, REQUEST_DEGRADE 700.
  - Degrade also needs confidence ≥ 80 % and 3 hits within 500 ms.
- **Why this split.**
  - Rules are deterministic and explainable, so they sit on the real-time path.
  - Isolation Forest needs no labelled attacks.
  - The safety task keeps the final say over the vehicle.

Block Diagram, Flowchart, Models, Results

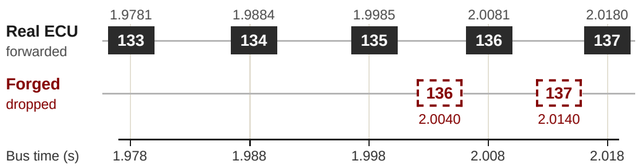


Protected ECUs on CAN-B

ECU	ID	Period
Brake	0x0A0	10 ms
Steering	0x0B0	10 ms
Powertrain	0x100	20 ms
Battery	0x200	100 ms
Body	0x300	100 ms
Infotainment	0x400	200 ms

Figure 1. System architecture. CAN-A frames reach CAN-B only through the STM32. The learned tier sees a copy, so the bus stays protected if the Pi fails.

(a) Timeline on ID 0x0A0



(b) Byte comparison: last trusted frame vs forged frame

	0 pressure	1	2 brake on	3 byte 3	4 byte 4	5 byte 5	6 counter	7 checksum
Last trusted 1.9985 s (real)	00	35	01	00	00	00	87	BD
Forged 2.0040 s	00	B2	01	00	00	00	88	3B

Only byte 1 (pressure) jumps: 35 → B2. The counter (87 → 88) and the checksum (3B) are both valid.

(c) The eight checks on the forged frame

Check	Expected	This frame	Result
Known ID	in routing table	0x0A0	Pass
Length	8 bytes	8 bytes	Pass
Checksum	3B	3B	Pass
Timing	≥ 5 ms since last	5.5 ms	Pass
Flood rate	< 3× normal	1.1×	Pass
Counter	136	136	Pass
Exact copy	new payload	new payload	Pass
Plausibility	5.3 ± 1.5 bar	**17.8 bar (Δ 12.5)**	**Fail**

(d) Decision

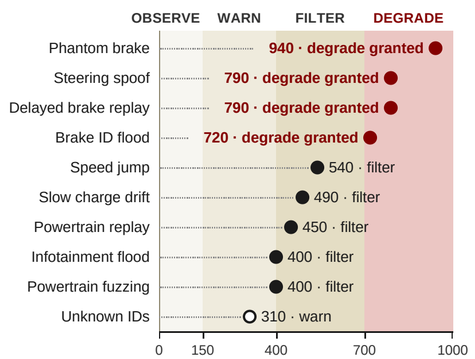
Implausible value **80** × Braking **95** / 100 = **76**
76 × confidence **85 %** = 64 → risk **640 / 1000**

Result: FILTER. The frame is dropped, and the next real frame (counter 136) passes.

Figure 2. Telling a forged brake frame from a real one. The forged frame carries the correct next counter and a valid checksum. It fails only the plausibility check: pressure jumps 12.5 bar in one frame against a learned limit of 1.5 bar.

Attack	Rule tier	Learned tier	Forged frames blocked	Real frames dropped	Highest response
Steering spoof	5/5	5/5	100.0 %	0.00 %	**Request degrade**
Phantom brake	5/5	5/5	86.1 %	3.77 %	**Request degrade**
Powertrain replay	5/5	5/5	100.0 %	0.00 %	Filter
Delayed brake replay	5/5	5/5	100.0 %	0.02 %	**Request degrade**
Infotainment flood	5/5	**0/5**	99.7 %	0.38 %	Filter
Brake ID flood	5/5	5/5	100.0 %	0.00 %	**Request degrade**
Powertrain fuzzing	5/5	5/5	100.0 %	0.00 %	Filter
Unknown IDs	5/5	**0/5**	100.0 %	0.00 %	Warn
Slow charge drift	**0/5**	5/5	0.0 %	0.00 %	Filter
Speed jump	5/5	5/5	97.3 %	0.00 %	Filter
Clean traffic (10 runs, 32,998 frames)	0 alerts/min	1 alert/min	n/a	0.00 %	None

Figure 3. Detection over 5 runs per attack. The learned tier was trained on clean traffic only.



Degrade also needs confidence ≥ 80 % and 3 hits within 500 ms.

Figure 4. Peak risk score per attack (one run each). Only brake and steering attacks cross 700 and earn a degrade request.

Conclusion

Discussion/Conclusion

- The rule tier caught 9 of 10 attacks on the first forged frame, with no false alarms and no real frames dropped across 32,998 clean frames.
- The learned tier caught the one attack no single frame reveals: a slow battery-charge drift, flagged about 0.9 s after it began.
- Brake and steering attacks escalated to a degrade request in every run, while infotainment attacks never went past Filter.

Limitations

- **Phantom brake.** When the forged pressure is close to the real one, the forged frame passes every check and claims the next counter first. 3.8 % of real frames were dropped.
- **False alerts.** The learned tier raises about 1 false alert per minute on clean traffic.
- **Simulated timing.** Times are simulated bus time, not STM32 measurements.

Future Direction

- **Hardware.** FreeRTOS on the STM32 F446RE: native bxCAN on CAN-A, MCP2515 on CAN-B, ESP32 as the attacker.
- **Real-bus tests.** Run the same ten attacks on the real bus, and measure latency, CPU and memory.
- **Authentication.** Message authentication (SecOC-style MAC, key in an ATECC608) on brake and steering IDs.

References and Affiliations

References

[1] Y. Jeong, H. Kim, S. Lee, W. Choi, D. H. Lee and H. J. Jo, "In-vehicle network intrusion detection system using CAN frame-aware features," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 5, pp. 3843–3853, 2024.

[2] C. Jichici, B. Groza, R. Ragobete, P.-S. Murvay and T. Andreica, "Effective intrusion detection and prevention for the commercial vehicle SAE J1939 CAN bus," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17425–17439, 2022.

[3] H. Sun, M. Sun, J. Weng and Z. Liu, "Analysis of ID sequences similarity using DTW in intrusion detection for CAN bus," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10426–10441, 2022.

[4] K. Wang, Q. Jiang, B. Wang, Y. Wu and H. Zhang, "StatGraph: Effective in-vehicle intrusion detection via multi-view statistical graph learning," *IEEE Trans. Intell. Transp. Syst.*, 2025.

Acknowledgements

We thank our supervisor Dr. T. Vijay Muni and project coordinator Dr. B. Jyothi, Department of Electrical and Electronics Engineering, KL University, for their guidance.